

Session 18 - Important Pandas DataFrame Methods

Data Science Mentorship Program (DSMP) 2022-23

1. value_counts()

Applicable to: Both Series and DataFrames.

Explanation

The `value_counts()` method is used to find the frequency of unique items in a column or a Series. It counts how many times each unique value appears.

Code (Series Example)

```
import pandas as pd
import numpy as np

# Creating a Series
s = pd.Series([1, 1, 1, 2, 2, 3])
print(s.value_counts())
```

Line-by-line Explanation:

1. `import pandas as pd`: Imports the Pandas library.
2. `s = pd.Series(...)`: Creates a Series with some repeating numbers (1 appears thrice, 2 appears twice).
3. `s.value_counts()`: Counts the frequency. The output will show 1 has 3 counts, 2 has 2, and 3 has 1.

Code (DataFrame Example)

```
# Assuming 'marks' is a DataFrame with student data
marks.value_counts()
```

Line-by-line Explanation:

1. `marks.value_counts()`: When applied to a whole DataFrame, it identifies duplicate rows and counts their frequency. If a row is identical to another, the count will be 2.

Practical Use Case (IPL Data)

Question: Find the player who has won the most "Man of the Match" awards in IPL finals and semi-finals.

```
# Filtering for special matches (Finals, Qualifiers, etc.)
# We use str.isdigit() because normal match numbers are digits, special ones are strings like 'Final'
import pandas as pd
ipl = pd.read_csv('ipl-matches.csv')

# Find rows where 'MatchNumber' is NOT a digit
mask = ~ipl['MatchNumber'].str.isdigit()
special_matches = ipl[mask]

# Get the player with most awards
result = special_matches['Player_of_Match'].value_counts()
print(result)
```

Line-by-line Explanation:

1. `ipl['MatchNumber'].str.isdigit()`: Checks if the match number is a digit (1, 2, 3...).
2. `~`: The "NOT" operator. We want rows where the match number is a string (e.g., 'Final').
3. `ipl[mask]`: Filters the DataFrame to keep only those special matches.
4. `['Player_of_Match'].value_counts()`: Counts how many times each player won the award in these filtered matches.

2. sort_values()

Applicable to: Both Series and DataFrames.

Explanation

This method is used to sort the data. You can sort in ascending (default) or descending order.

Code (Single Column)

```
# Sorting movies by title
movies.sort_values('title_x', ascending=False)
```

Line-by-line Explanation:

1. `movies.sort_values('title_x', ...)`: Sorts the entire DataFrame based on the 'title_x' column.
2. `ascending=False`: Sorts in Z to A order (descending).

Code (Handling Missing Values)

```
# Sorting while placing NaNs at the top
students.sort_values('name', na_position='first')
```

Line-by-line Explanation:

1. `na_position='first'`: By default, missing values (NaN) go to the end. This parameter brings them to the top.

Code (Multiple Columns)

```
# Sort by Year (Ascending) and then Title (Descending)
movies.sort_values(['year_of_release', 'title_x'], ascending=[True, False])
```

Line-by-line Explanation:

1. `['year_of_release', 'title_x']`: First, it sorts by year. If two movies have the same year, it then sorts those specific movies by title.
2. `ascending=[True, False]`: Applies 'True' (Asc) to the first column and 'False' (Desc) to the second.

3. rank()

Applicable to: Series Only.

Explanation

The `rank()` function assigns a numerical rank to values. In a classroom, a student with the highest marks gets Rank 1. In Pandas, by default, the lowest value gets Rank 1 (ascending).

Code (IPL Batsman Ranking)

```
# Ranking batsmen by runs (Highest runs = Rank 1)
batsman['rank'] = batsman['batsman_run'].rank(ascending=False)
batsman.sort_values('rank')
```

Line-by-line Explanation:

1. `batsman['batsman_run'].rank(...)`: Calculates rank based on runs.
2. `ascending=False`: Ensures the highest runner gets rank 1.0.
3. `batsman['rank'] = ...`: Stores this in a new column.

Important Note: If two players have the same runs, Pandas gives them the same rank (e.g., 5.5).

4. set_index() & reset_index()

Applicable to: DataFrames.

Explanation

- **set_index:** Turns a column into the index (row labels).
- **reset_index:** Turns the index back into a normal column and creates a default numeric index.

Code

```
# Making 'batsman' name the index
batsman.set_index('batsman', inplace=True)
```

```
# Resetting back
batsman.reset_index(inplace=True)
```

Key Point: Using `reset_index()` on a **Series** converts it into a **DataFrame**. This is a very common trick in data analysis.

5. unique() & nunique()

Applicable to: Series/Columns.

Explanation

- **unique():** Returns an array of all distinct values (including NaN).
- **nunique():** Returns the **count** of unique values.

Code

```
# All unique seasons in IPL
ipl['Season'].unique()
```

```
# Count of unique seasons
ipl['Season'].nunique()
```

Crucial Difference: `nunique()` does **not** count NaN (missing values) by default, while `unique()` shows them.

6. isnull(), notnull() & hasnans

Applicable to: Both.

Explanation

These are used to detect missing values (NaN).

- `isnull()`: Returns True if value is missing.
- `notnull()`: Returns True if value is **not** missing.
- `hasnans`: A property (Series only) that returns a single True/False if the column contains any NaN.

7. dropna()

Applicable to: Both.

Explanation

Used to remove rows or columns that contain missing values.

Code

```
# Drop rows where ALL values are missing
students.dropna(how='all')
```

```
# Drop rows where 'name' or 'college' is missing
students.dropna(subset=['name', 'college'])
```

Line-by-line Explanation:

1. `how='all'`: Only deletes the row if every single column in that row is empty.
2. `subset=['name']`: Only looks at the 'name' column. If 'name' is missing, the row is deleted, even if other columns are filled.

8. fillna()

Applicable to: Both.

Explanation

Instead of deleting missing data, we fill it with a specific value (e.g., 0, "Unknown", or the Average).

Code

```
# Fill missing names with "Unknown"
students['name'].fillna('Unknown', inplace=True)
```

```
# Fill missing runs with the Mean (Average)
batsman['runs'].fillna(batsman['runs'].mean())
```

```
# Forward Fill (copies value from the previous row)
students['name'].fillna(method='ffill')
```

9. drop_duplicates()

Applicable to: Both.

Explanation

Removes identical rows.

Code

```
# Keep only the last occurrence of a duplicate
marks.drop_duplicates(keep='last')

# Drop based on specific columns
ipl.drop_duplicates(subset=['Season', 'City'], keep='last')
```

10. apply()

Applicable to: Both.

Explanation

apply() is used to execute a custom function on every element of a Series or every row/column of a DataFrame.

Code (Series)

```
# Applying a custom 'sigmoid' math function to a column
def sigmoid(val):
    return 1 / (1 + np.exp(-val))

marks['points'].apply(sigmoid)
```

Code (DataFrame - Row-wise)

```
# Calculating distance between two points (stored in different columns)
def calculate_dist(row):
```

```
return (row['p1']**2 + row['p2']**2)**0.5
```

```
df.apply(calculate_dist, axis=1)
```

Line-by-line Explanation:

1. `def calculate_dist(row)`: The function receives a whole row as an object.
2. `row['p1']`: Accesses the value of 'p1' for that specific row.
3. `axis=1`: **Very Important!** This tells Pandas to apply the function **row-wise** instead of column-wise.

Tips & Tricks Summary

1. **Permanent Changes**: Methods like `sort_values`, `set_index`, and `dropna` do not change the original data unless you use `inplace=True`.
2. **Series to DataFrame**: Use `reset_index()` on a Series to instantly turn it into a table (DataFrame).
3. **Frequency Analysis**: Always use `value_counts()` to quickly understand the distribution of categorical data.
4. **Chaining**: You can combine methods: `df.isnull().sum()` tells you the count of missing values in each column.

End of Notes